



Projecto Final

Programação Orientada a Objectos, Estruturas de Dados e Manipulação de Ficheiros

1. Introdução

Neste projecto, pretende-se a implementação de uma aplicação completa em linguagem Java, sem componente gráfica para interação com o utilizador, recorrendo ao paradigma de Programação Orientada a Objectos.

A aplicação tem como objectivo auxiliar o funcionamento de uma clínica de análises médicas na gestão dos exames laboratoriais realizados.

Espera-se o desenvolvimento e aplicação de conhecimentos e competências relacionadas com as temáticas abordadas na disciplina, com especial incidência para:

- **programação orientada a objectos**;
- **estruturas de dados**;
- **manipulação de ficheiros** para armazenamento de dados de forma persistente;
- bem como o desenvolvimento de **interfaces para interação com o utilizador** (em modo texto).

2. Objectivos

Existe um conjunto de requisitos que o projecto deve respeitar e implementar (os requisitos com a etiqueta **Exame** apenas se aplicam às Épocas da Avaliação por Exame, não sendo desenvolvidos durante a Avaliação Periódica):

2.1. Descrição global da aplicação

Gestão de acesso e utilizadores

- [**R1**] Permitir os utilizadores registarem-se e autenticarem-se na aplicação, podendo a aplicação ser usada por vários utilizadores simultaneamente.
- [**R2**] Permitir o acesso à aplicação por 3 tipos de utilizadores: administradores, técnicos e clientes.
- [**R3**] Os **utilizadores** são caracterizados pelos atributos username, password, nome, estado (activo/inactivo), email e tipo (administradores, técnicos ou clientes).
- [**R4**] Cada utilizador apenas pode alterar a sua própria informação, não podendo alterar ou visualizar dados de outros utilizadores, ou criar utilizadores.
- [**R5**] O username e email devem ser únicos.
- [**R6**] Os **clientes** e **técnicos** caracterizam-se adicionalmente por número de identificação fiscal (NIF), morada e contacto telefónico.
O NIF e contacto telefónico são únicos.
- [**R7**] Caso não existam utilizadores criados, a aplicação deve solicitar a criação de um utilizador (administrador).
- [**R8**] Após a autenticação, a aplicação deve apresentar a mensagem

“Bem-vindo [*nome utilizador*]”.

[R9] Quando a aplicação estiver a encerrar, deve apresentar a mensagem

“Adeus [*nome utilizador*]”.

Funções

[R10] Os **administradores** aprovam os pedidos de registo dos utilizadores. Todos os pedidos devem ser aprovados antes de poderem ser usados para autenticação.

[R11] Os **administradores** associam um técnico responsável a cada serviço.

[R12] Os **técnicos** introduzem análises laboratoriais, categorias, testes, áreas médicas e fornecedores, e encomendas.

[R13] Os **clientes** realizam pedidos de serviços de análises laboratoriais, que devem ser aprovadas pelos técnicos.

[R14] Os **administradores** podem gerir todos os serviços.

[R15] Os **técnicos** apenas podem gerir os seus serviços realizados.

[R16] Os **clientes** apenas podem gerir os seus serviços pedidos.

Acções

[R17] O procedimento inicia com o **cliente** a submeter um pedido de serviço.

[R18] O **administrador** deve aprovar ou rejeitar o serviço, associando um técnico responsável.

[R19] Um **serviço** caracteriza-se um código, a listagem das análises laboratoriais a realizar, o valor total do serviço e o estado (se está aprovada, ou ainda aguarda aprovação ou concluído).

Adicionalmente, os serviços devem possuir uma data em que foram pedidos pelo cliente e uma data em que foram concluídos pelo laboratório e um estado.

[R20] O **serviço** deve modificar o seu estado ao longo do processo:

- a) **iniciado** – quando o cliente realiza;
- b) **aprovado** – quando o administrador aprova o pedido;
- c) **rejeitado** – quando o administrador rejeita ou anula o pedido;
- d) **execução** – quando o técnico responsável inicia o tratamento do pedido
- e) **terminado** – quando o técnico conclui o tratamento e o serviço é remetido ao cliente.

[R21] Deve ser possível introduzir novas **análises laboratoriais**, que posteriormente podem ser associadas a vários serviços.

Cada análise é definida por um código interno, designação genérica, certificação associada (A, B, C, D ou E), laboratório que certifica, técnico que realiza a análise, técnico superior que supervisiona e componentes químicos necessários para a análise.

Uma análise possui um conjunto indeterminado de testes. Os testes só são acedidos por um técnico isoladamente, em cada acesso.

Uma análise deve possuir a data em que foi terminada e resultado final.

[R22] Cada serviço possui um técnico responsável.

[R23] Esse técnico responsável deve indicar o técnico por cada análise.

[R24] O código de uma análise e de um serviço devem ser únicos.

[R25] Cada **análise laboratorial** deve ter associado uma lista até 6 fornecedores dos componentes químicos necessários, podendo ser introduzido um novo fornecedor a qualquer momento.

[R26] Cada **fornecedor** é caracterizado por nome, morada e contacto telefónico.

[R27] Cada **análise laboratorial** deve apresentar uma lista de até 4 áreas médicas (ou menos), podendo ser introduzida uma nova análise laboratorial a qualquer momento.

- [**R28**] Cada **área médica** é caracterizada pela sua designação e família.
- [**R29**] Cada **certificação** possui um nível e grau.
- [**R30**] Cada **componente químico** é caracterizado pela designação, valor alfa e valor beta (atributos alfanuméricos) e código único.
Cada componente químico possui uma quantidade em *stock* que o laboratório armazena.
- [**R31**] Os técnicos encomendam componentes químicos a fornecedores específicos.
Cada **encomenda** possui a lista de componentes químicos, quantidade encomendada, data do pedido, data de entrega e código único.
- [**R32**] Quando é realizada a compra de componentes químicos aos fornecedores, a respectiva quantidade em *stock* em armazém deve ser incrementada.
- [**R33**] Cada **teste** é definido pela designação, pelo valor medido, valor de referência e unidade de medida.
- [**R34**] É possível acrescentar ou remover testes a realizar numa análise.
- [**R35**] (**Exame**) Deve ser emitido um **recibo** ao cliente, indicando o NIF e custo total do serviço realizado.
- [**R36**] (**Exame**) Deve existir um **módulo de estatísticas** que indica o número e valor total de serviços solicitados por um cliente, o tempo médio de execução dos serviços, o número médio de análises por serviço e o número médio de componentes químicos por análise.
- [**R37**] (**Exame**) O sistema deve notificar no início da sessão do administrador para encomendas ou serviços realizados há mais de 10 dias sem ainda terem sido concluídos.

Listagens e pesquisas

- [**R38**] Deve ser possível ordenar **utilizadores** por ordem alfabética do nome.
- [**R39**] (**Exame**) Deve ser possível ordenar **serviços** por ordem da data do pedido (do mais recente ao mais antigo).
- [**R40**] Deve ser possível ordenar **serviços** por código.
- [**R41**] (**Exame**) Deve ser possível ordenar **serviços** por número (quantidade) de análises.
- [**R42**] Deve ser possível ordenar **serviços** por valor total.
- [**R43**] Deve ser possível ordenar **análises** por código.
- [**R44**] (**Exame**) Deve ser possível ordenar **análises** por número (quantidade) de testes.
- [**R45**] Deve ser possível ordenar **encomendas** por código.
- [**R46**] (**Exame**) Deve ser possível ordenar **encomendas** por ordem da data de pedido (do mais recente ao mais antigo).

- [**R47**] Deve ser possível listar todos os **utilizadores**.
- [**R48**] Deve ser possível listar **utilizadores** por tipo.
- [**R49**] Deve ser possível listar todos os **serviços**.
- [**R50**] Deve ser possível listar todos os **serviços** associados a um cliente.
- [**R51**] Deve ser possível listar todos os **serviços** por estado (por qualquer um dos estados possíveis).
- [**R52**] Deve ser possível listar todos os **serviços** que possuem uma análise específica.
- [**R53**] Deve ser possível listar todos os **serviços** onde foi usado um componente químico específico.
- [**R54**] (**Exame**) Deve ser possível listar **serviços** com tempo de execução superior a um determinado limite (introduzido pelo utilizador no momento de pesquisa).
- [**R55**] Deve ser possível listar todas as **análises**.
- [**R56**] (**Exame**) Deve ser possível listar as **análises** mais solicitadas (e.g. 10 análises mais solicitadas por clientes).
- [**R57**] Deve ser possível listar **testes** realizados por análise.

- [R58] Deve ser possível listar todos os **componentes químicos**.
- [R59] (Exame) Deve ser possível listar **componentes químicos** com uma quantidade de stock abaixo de um determinado limite (introduzido pelo utilizador no momento de pesquisa).
- [R60] Deve ser possível listar todas as **encomendas**.
- [R61] Deve ser possível listar todas as **encomendas** realizadas, mas não entregues.

- [R62] Deve ser possível pesquisar **utilizadores** por username ou nome.
- [R63] Deve ser possível pesquisar **serviços** por código ou uma palavra que surja na descrição.
- [R64] Deve ser possível pesquisar **análises** por código.
- [R65] Deve ser possível pesquisar **análises** que usaram um determinado componente químico.

- [R66] Deve ser possível realizar pesquisas avançadas, ou seja, apresentar todos os registos que apresentem um termo de pesquisa, mesmo que parcialmente

(e.g. termo de pesquisa “Ana” deve apresentar como resultado “Ana Sousa”, “Ana Silva” e “Anabela”).

- [R67] Os **clientes** podem listar e pesquisar os seus serviços que realizaram.
- [R68] Os **técnicos** podem listar e pesquisar os seus serviços que processaram.
- [R69] Os **administradores** podem listar e pesquisar todos os serviços.

2.2. Manipulação e armazenamento de dados persistente

- [R70] O acesso à aplicação deve ser restringido com credenciais (*username/password*), informação que deverá ser armazenada num ficheiro de texto “credenciais_acesso.txt”.
- [R71] Durante o encerramento da aplicação, os dados devem ser automaticamente guardados num ficheiro de objectos “dados_apl.dat”.
- [R72] A aplicação, no arranque, deve automaticamente ler os dados do ficheiro de objectos “dados_apl.dat”, caso este exista, e informar o utilizador que os dados foram lidos com sucesso.

Caso o ficheiro esteja indisponível deve surgir uma mensagem a informar a situação.
- [R73] Deve ser possível exportar todos os serviços realizados para um ficheiro de texto em formato CSV (*Comma Separated Values*), com a estrutura

<data>, <valor>, <cliente>, <análises>

ordenados do serviço mais recente para o mais antigo.
- [R74] (Exame) Deve ser possível exportar todas as análises e para cada análise todos os testes realizados nos vários serviços, para um ficheiro de texto em formato CSV (*Comma Separated Values*), com a estrutura

<analise>, <data>, <lista de testes>

ordenados alfabeticamente.

Monitorização de acessos

- [R75] Todas as acções dos utilizadores deverão ser guardadas num ficheiro de texto denominado “log.txt”.

Este ficheiro deverá ser escrito de forma sequencial, apresentando as acções mais recentes no início do ficheiro.

As entradas deverão ter o seguinte formato: <utilizador> <acção>.
- [R76] Deve ser possível consultar o *log* de acções através da aplicação.

- [R77] Deverá ser registado o número total (até ao momento) de execuções do sistema e o *username* do último utilizador que acedeu à aplicação.

Esta informação deverá ser guardada num ficheiro de objectos e actualizada sempre que a aplicação é executada, apresentando os dados ao utilizador.

Quando a aplicação é terminada esta informação deve ser novamente armazenada no ficheiro, sendo este denominado "info_sistema.dat".

2.3. Gestão geral da aplicação

Interacção com o utilizador

- [R78] Disponibilizar uma interface em modo texto onde o utilizador possa interagir e controlar a aplicação.

Programação Orientada a Objectos

- [R79] A aplicação deve estar correctamente estruturada, tendo em conta o paradigma Orientado a Objectos e recorrendo à linguagem Java.

- [R80] Não é permitido o uso de função anónimas, funções *lambda* ou *streams*.

Na listagem de estruturas dinâmicas não devem ser usados ciclos *for* / *foreach*, mas iterações através das estrutura *Enumeration* ou *Iterator*.

- [R81] Implemente as estruturas de armazenamento necessárias, procurando optimizar os recursos utilizados.

Validação de dados e notificações

- [R82] Valide todas as leituras de dados do utilizador (e.g. verifique se os nomes são únicos).

- [R83] Sempre que necessário, apresentar ao utilizador mensagens informativas adequadas.

Quando um utilizador realizar uma acção sobre a aplicação, esta deve informar se acção foi realizada com sucesso ou insucesso.

3. Implementação

O programa deve ser implementado na linguagem Java. Lembre-se que é uma linguagem orientada a objectos, pelo que deverá ter em conta os seguintes aspectos:

- Cada classe deve gerir internamente os seus dados, pelo que deverá cuidar da protecção das suas variáveis e métodos.
- Cada objecto deverá ser responsável por uma tarefa ou objectivo específico, não lhe devendo ser atribuídas funções indevidas.
- Utilize a *keyword static* apenas quando tal se justifique e não para evitar erros do compilador.
- Recomenda-se que elabore um diagrama com as suas classes e objectos antes de iniciar o projecto, para prever a estrutura do projecto.

Tenha ainda em conta os seguintes pontos que serão importantes na avaliação:

- Comente as classes, métodos e variáveis públicas (não é obrigatório gerar Javadoc). Comente o restante código sempre que a leitura dos algoritmos não seja óbvia
- Comente o restante código sempre que a leitura dos algoritmos não seja óbvia.
- Tal como sugerido acima, evite o uso abusivo de *static* e de variáveis e métodos *public*.
- Na escolha de nomes para variáveis, classes e métodos siga as convenções adoptadas na linguagem Java.
- Procure uma interface agradável com o utilizador (na obtenção de dados e disponibilização de informação). As entradas de dados por parte do utilizador deverão ser testadas e protegidas contra erros ou falhas que possam surgir.

4. Logística

Prazos e procedimentos de submissão

- Durante o presente ano lectivo apenas existe **um único projecto a ser desenvolvido durante o período de leccionação**, independentemente da época de avaliação a que o estudante se submete (periódica ou exame).
- O projecto pode ser desenvolvido **individualmente ou em grupo (máximo 2 elementos)** na Avaliação Periódica. Na Época de Avaliação por Exame, o projecto deve ser desenvolvido individualmente.
- O projecto tem um peso de 40% (08 valores em 20) na avaliação final da unidade curricular.
- O projecto tem como **prazo final de entrega a véspera da data para a avaliação da época normal de Programação a definir no mapa de avaliações, até às 12h00** para todas as épocas.
Nota: Não tendo ainda sido divulgado pelos órgãos da escola este mapa oficial, a data encontra-se provisoriamente agendada para 22 de Janeiro, sendo actualizada assim que o mapa for aprovado e divulgado.
- Será submetido obrigatoriamente um **relatório intermédio até ao dia 20 de Dezembro, 23h00** (apenas para Avaliação Periódica, não aplicável à Avaliação por Exame, Época Normal), respeitando o modelo disponibilizado.
- Serão agendadas **apresentações dos projectos e defesas individuais**,
 - Cada estudante/grupo deve **reservar um bloco após a submissão final do projecto**. Os blocos de defesa não podem ser reservados antes da submissão do trabalho.
 - Nas defesas será solicitado que os estudantes expliquem e alterem o código submetido, bem como implementem novo código presencialmente para novos requisitos.
 - Desrespeito pelas regras, não comparência ou reprovação na defesa implica anulação do trabalho e reprovação à avaliação.
- Submissão final implica a entrega, em formato digital, das seguintes componentes:
 - **código da aplicação** (ficheiros com extensão `.java`, incluindo ficheiros adicionais relevantes);
 - **relatório final** seguindo obrigatoriamente o modelo disponibilizado, onde seja documentada a aplicação, descrevendo-se as classes manipuladas, bem como as estruturas de dados; algoritmos ou soluções de implementação seleccionadas e respectivas justificações;
 - **manual do utilizador**, instruindo o utilizador de como poderá interagir com a aplicação.
- Todo o material deve ser agrupado num único ficheiro `.rar` ou `.zip`.
- O ficheiro do projecto submetido deve respeitar a nomenclatura seguinte:
`LEI_Prog_2025_AP3_Nome-Apelido_2026-01-xx.rar` (Avaliação Periódica), ou
`LEI_Prog_2025_AE_Nome-Apelido_2026-01-xx.rar` (Todas as épocas da Avaliação Exame).
- A entrega implica a **submissão exclusivamente através da plataforma Nonio**. Nenhum trabalho será aceite fora desta plataforma ou após terminar o prazo definido (a plataforma de submissão encerrará automaticamente);
- O desenvolvimento ocorrerá tanto em períodos de aulas como extra-aulas.
- Sendo o foco do projecto avaliar as competências em programação do estudante, **todo o código deve ser desenvolvido exclusivamente pelo estudante**. Não são permitidas ferramentas de geração automática de código e/ou motores de Inteligência Artificial (e.g. *CoPilot*, *ChatGPT*, *IDE Code Completion*), nem código produzido por outros elementos. Projectos que incluam código ou indícios de código não desenvolvido pelo estudante serão considerados fraude.
- Situações de **fraude (cópia/plágio)** implicam a **imediata anulação da avaliação (zero valores) e sancionados de acordo com o Estatuto Disciplinar do Estudante**. A cópia de programas resultará na atribuição de nota zero a todos os alunos envolvidos. Também não é permitido copiar código da internet não referenciado.
- Todas as **fontes externas** (e.g. livros, páginas de internet, recursos didáticos, ferramentas) devem ser explicitamente **referenciadas**, seguindo o formato indicado no modelo de relatório.

- Os estudantes devem ter em consideração que apenas serão aceites para avaliação projectos que:
 - incluam **todos os elementos de avaliação obrigatórios**,
 - possuam **pelo menos 50% dos requisitos totalmente implementados** (das secções *Descrição Global da Aplicação e Manipulação e Armazenamento de Dados Persistente*),
 - na avaliação por Exame **todos os requisitos** indicados com a etiqueta **Exame** têm de estar **implementados**,
 - a aplicação **execute integralmente** sem anomalias,
 - a aplicação **seja capaz de ler e escrever em todos os ficheiros de texto e de objectos** definidos no enunciado, e
 - a **totalidade do código é da autoria exclusiva do estudante**, sendo **totalmente proibido incluir código produzido por ferramentas de geração automática de código** (e.g. CoPilot, ChatGPT, IDE Code Completion).

O não cumprimento de um dos requisitos assinalados **implica a anulação do trabalho e consequentemente da avaliação**.

Documentação anexa

- **O relatório final deve seguir obrigatoriamente e rigidamente o modelo disponibilizado:**
 - **Todas as secções do modelo são obrigatórias** e devem ser adequadamente preenchidas. O modelo deve assim ser respeitando na totalidade.
 - **Não podem ser eliminadas do documento secções** e toda a imagem tem de ser acompanhada de legenda e de um texto a descrever e explicar essa imagem.
 - Secções removidas, sem conteúdo, apenas com imagens, ou com texto genérico implicam a anulação do documento.
 - O documento deve possuir um conjunto de **referências obrigatórias**. Todo o material e fontes de apoio e consulta para a elaboração do projecto devem ser incluídas. A não inclusão desta secção implica a anulação do documento.
 - Tal como no desenvolvimento do código do projecto, o relatório deve ser elaborado exclusivamente pelo estudante, **não sendo permitido o uso de ferramentas de geração de texto**, nomeadamente ferramentas de IA.
 - A anulação do relatório, sendo um elemento de submissão obrigatória, **implica a reprovação à avaliação** no presente ano letivo (não é possível nova submissão de projecto).

O não cumprimento de um dos requisitos assinalados **implica a anulação do relatório e consequentemente do relatório**.

- **O relatório deve conter, entre outros, os seguintes tópicos** (não de forma exclusiva, sendo recomendável a inclusão de tópicos adicionais sempre que estes permitam uma melhor clarificação do projecto e tornem o documento mais completo):
 - Descrição dos objectivos a atingir com o projecto;
 - **Indicação explícita dos requisitos implementados e não implementados**, justificando os requisitos incompletos (recomenda-se o uso de uma tabela listando todos os requisitos e o seu estado);
 - **Distribuição das tarefas ao longo do tempo** - indicação do planeamento das tarefas e recursos utilizados (com especial foco no tempo, referindo o tempo gasto em cada requisito). Em caso de trabalhos conjuntos explicitamente referir que requisitos foram realizados por cada elemento e qual o tempo dedicado;
 - **Descrição das classes e respectivos métodos**;
 - Apresentação dos **algoritmos** adoptados;
 - Indicação das **estruturas de dados** implementadas, justificando as escolhas realizadas;

- **Testes** realizados para averiguar a completude dos requisitos e o correcto funcionamento da aplicação;
 - Avaliação crítica do projecto: **pontos fortes** e **pontos fracos**;
 - Possíveis **melhorias futuras**;
 - **Referências bibliográficas** (material e fontes consultadas).
- Em anexo:
- **Manual do utilizador** (descrição do funcionamento da aplicação indicando os comandos disponíveis, como executá-los e obtenção de resultados);
 - Documentação adicional considerada relevante para o projecto.

Parâmetros de avaliação

- O código e execução da aplicação apresenta um peso de 70%, enquanto a documentação (relatório e manual) apresenta um peso de 30%.
- A avaliação dos trabalhos terá em conta vários parâmetros, nomeadamente:
 - Programação Orientada a Objectos;
 - Usabilidade da interacção com o utilizador;
 - Manipulação de ficheiros e de estruturas de dados relacionais;
 - Completude dos requisitos;
 - Funcionalidades disponíveis;
 - Protecções implementadas, tanto na leitura de dados do utilizador, como no acesso a ficheiros;
 - Interacção com o utilizador;
 - Tratamento de excepções;
 - Execução da aplicação sem erros;
 - Respeito pelas normas de codificação;
 - Qualidade do relatório;
 - Documentação do código;
 - Pesquisa e aplicação de novos conhecimentos;
 - Domínio da linguagem java;
 - Qualidade do relatório;
 - Qualidade do manual do utilizador;
 - Apresentação e defesa do trabalho realizado;
 - Capacidade de explicar e alterar o código submetido.